Expt. No. 3

Title: Implement Perceptron algorithm to simulate any logic gate.

Rubric	***Score (0 to 5)***
Knowledge	
Delivery	
Output and Conclusion	
Total	

Performed On: 25/08/2025

***Sign:*_____**

Experiment 3

Aim: ***Implement Perceptron algorithm to simulate any logic gate.***

Software: Google Collab

Pre Lab-Questions:

What do you understand by a Perceptron Neuron?

A perceptron neuron is a computational model that sums weighted inputs, applies a bias, and passes the result through an activation function to produce an output.

Post lab Questions:

What are the components of Single Layer Perceptron?

The components of a single-layer perceptron are inputs, weights, bias, summation function, activation function, and output.

Theory:

Overview

Perceptron is a linear supervised machine learning algorithm. It is used for binary classification. This article will introduce you to a very important binary classifier, the perceptrons, which forms the basis for the most popular machine learning models nowadays – the neural networks.

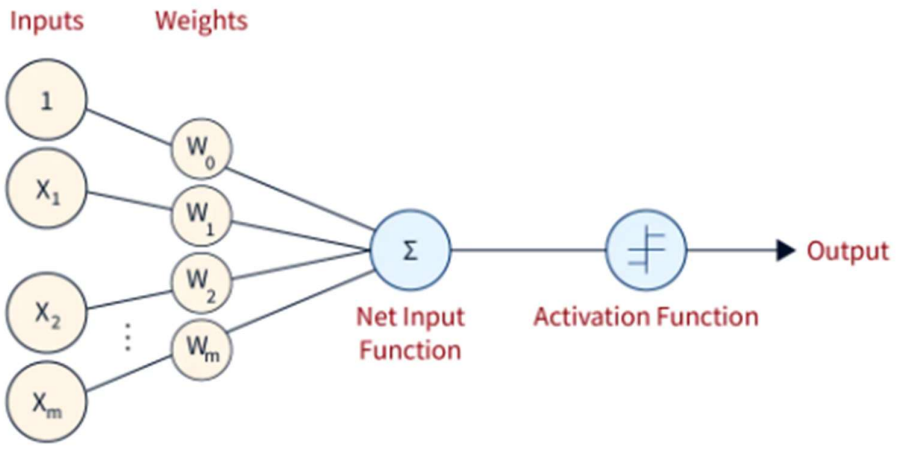
Introduction

Perceptron Learning Algorithm is also understood as an Artificial Neuron or neural network unit that helps to detect certain input data computations in business intelligence. The perceptron learning algorithm is treated as the most straightforward Artificial Neural network. It is a supervised learning algorithm of binary classifiers. Hence, it is a single-layer neural network with four main parameters, **i.e.**, input values, weights and Bias, net sum, and an activation function.

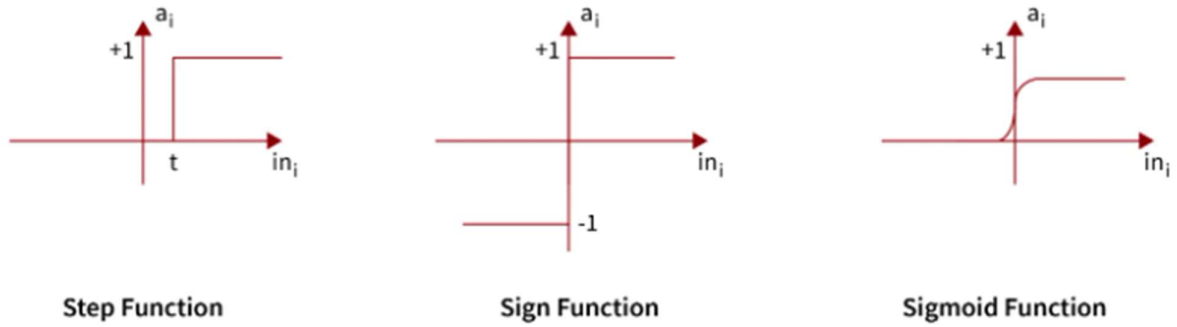
Working:

Basic Components of Perceptron

Frank Rosenblatt invented the perceptron learning algorithm.



It is a binary classifier and consists of three main components. These are:



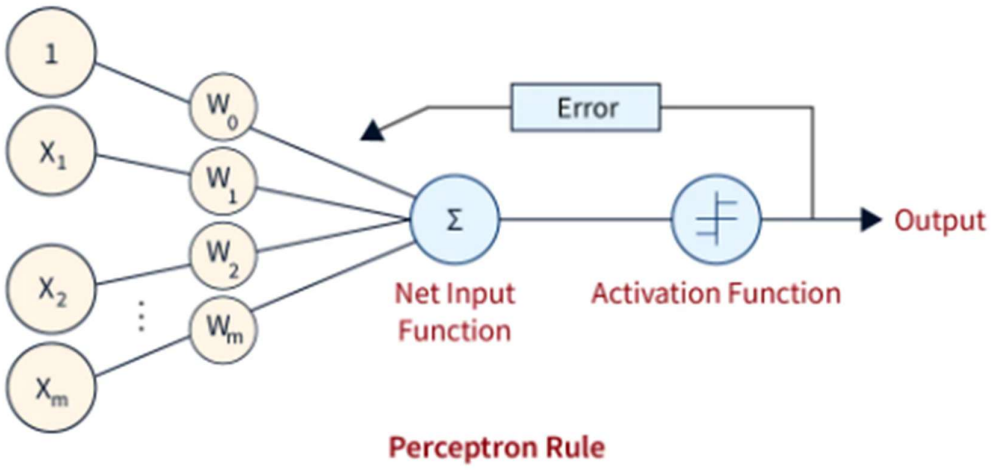
Types of Perceptron Models

Based on the number of layers, perceptron is broadly classified into two major categories:

1. **Single Layer Perceptron Model:**
   It is the simplest Artificial Neural Network (ANN) model. A single-layer perceptron model consists of a feed-forward network and includes a threshold transfer function for thresholding on the Output. The main objective of the single-layer perceptron model is to classify linearly separable data with binary labels.
2. **Multi-Layer Perceptron Model:**
   The multi-layer perceptron learning algorithm has the same structure as a single-layer perceptron but consists of an additional one or more hidden layers, unlike a single-layer perceptron, which consists of a single hidden layer. The distinction between these two types of perceptron models is shown in the Figure below.

Algorithm:

An overview of this algorithm is illustrated in the following Figure:



1. **Input Nodes or Input Layer:** Primary component of Perceptron learning algorithm, which accepts the initial input data into the model. Each input node contains an actual value.
2. **Weight and Bias:** The weight parameter represents the strength of the connection between units. Bias

can be considered as the line of intercept in a linear equation.

3. **Activation Function:** Final and essential components help determine whether the neuron will fire. The activation function can be primarily considered a step function. There are various types of activation functions used in a perceptron learning algorithm. Some of them are the sign function, step function, sigmoid function, etc.

Perceptron Function

Perceptron learning algorithm function f(x)$f(x)$ is represented as the product of the input vector (x) and the learned weight vector (w). In mathematical notion, it can be described as:
f(x)=1,ifw.x+b>0$f(x)=1,ifw.x+b>0$ f(x)=0,otherwise$f(x)=0,otherwise$

Where–

- w represents the weight vector which consists of a set of real-valued weights.
- b represents the bias vector.
- x represents the input vector which consists of the input feature values.

Geometry of the Solution Space

In the previous section, we learned about the weight update rules for the perceptron learning algorithm. We have already established that when x belongs to P, we want w.x > 0. That means that the angle between w and x should be less than 90 because the cosine of the slope is proportional to the dot product.

So, whatever the w vector may be, as long as it makes an angle less than 90 degrees with the positive example data vectors (x ∈∈ P) and an angle more than 90 degrees with the negative example data vectors (x ∈∈ N), we are cool.

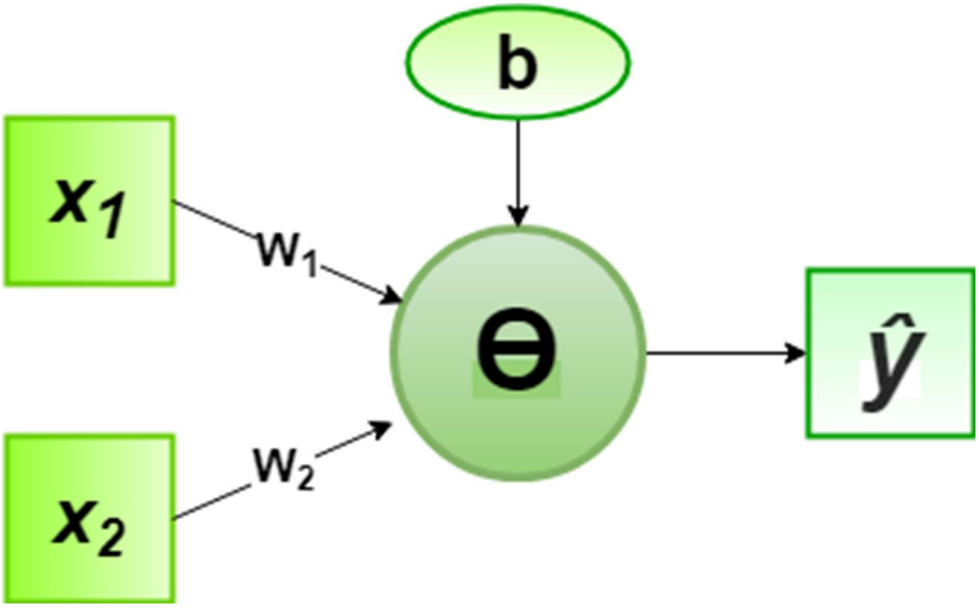
In the field of Machine Learning, the Perceptron is a Supervised Learning Algorithm for binary classifiers. The Perceptron Model implements the following function:

$$\begin{aligned}\hat{y} &= \Theta\left(w_1 x_1 + w_2 x_2 + \ldots + w_n x_n + b\right) \\ &= \Theta(\mathbf{w} \cdot \mathbf{x} + b) \\ \text{where } \Theta(v) &= \begin{cases} 1 & \text{if } v \geqslant 0 \\ 0 & \text{otherwise} \end{cases}\end{aligned}$$

X1	X2	Y
0	0	0
0	1	0
1	0	0
1	1	1

Now for the corresponding weight vector
$w : (w_1, w_2)$
of the input vector
$x : (x_1, x_2)$
, the associated Perceptron Function can be defined as:

$$\hat{y} = \Theta\left(w_1 x_1 + w_2 x_2 + b\right)$$



For the implementation, considered weight parameters are
$w_1 = 1, w_2 = 1$
and the bias parameter is
$b = -1.5$

Program & Output:

```
Import numpy as np

def unitStep(v):
  if v>=0:
    return 1
  else:
    return 0

def perceptronModel(x,w,b):
  v = np.dot(w, x) + b
  y = unitStep(v)
  return y

def AND_logic(x):
  w = np.array([1,1])
  b = -1.5
  return perceptronModel(x,w,b)


test1 = np.array([0, 1])
test2 = np.array([1, 1])
test3 = np.array([0, 0])
test4 = np.array([1, 0])

print(f"AND(0, 1) = {AND_logic(test1)}")
print(f"AND(1, 1) = {AND_logic(test2)}")
print(f"AND(0, 0) = {AND_logic(test3)}")
print(f"AND(1, 0) = {AND_logic(test4)}")
```

OUTPUT:

```
AND(0, 1) = 0
AND(1, 1) = 1
AND(0, 0) = 0
AND(1, 0) = 0
```

Conclusion:

The Perceptron algorithm effectively simulates logic gates like AND by learning the appropriate weights and threshold through training. This demonstrates how simple neural networks can model basic binary functions.

CO's Covered:

CO1. Implement basic neural network models.